

LAPORAN PRATIKUM STRUKTUR DATA

Singel Linked Lis



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

KATA PENGANTAR

Puji dan syukur kita panjatkan kehadiran allah yang maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil pratikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada bapak Wahyudi selaku dosen dalam mata kuliah pratikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan pratikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang pratikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	3
BAB II.....	5
PEMBAHASAN	5
2.1 Pengertian Single Linked List	5
2.2 Alat dan Bahan.....	5
2.3 Langkah langkah	5
BAB III	21
KESIMPULAN.....	21
3.1 Ringkasan Hasil Praktikum	21
3.2 Saran Pengembangan.....	21
DAFTAR PUSTAKA	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data merupakan hal yang sangat penting untuk menghasilkan program yang efisien dan terstruktur. Salah satu struktur data yang sering digunakan adalah linked list, khususnya singel linked list. Struktur ini memungkinkan penyimpanan data secara dinamis dibandingkan dengan array yang bersifat statis.

Singel linked list digunakan karena kemampuannya dalam menambah dan menghapus elemen tanpa perlu menggeser data lain seperti pada array. Oleh karena itu, pemahaman tentang konsep dan implementasi single linked list menjadi penting bagi mahasiswa dalam mempelajari struktur data.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar dari single linked list
2. Mampu mengimplementasikan single linked list dalam Bahasa pemrograman
3. Mengetahui cara operasi dasar seperti:
 - Menambah data (insert)
 - Menghapus data (delete)
 - Menampilkan data (display)
4. Melatih logika pemrograman dalam pengelolaan data dinamis.

1.3 Manfaat Praktikum

Adapun manfaat dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami perbedaan antara struktur data statis dan dinamis.
2. Meningkatkan kemampuan dalam mengelola data menggunakan pointer atau referensi

3. Menjadi dasar untuk memahami struktur data yang lebih kompleks seperti double linked list, stack dan queue
4. Membantu dalam pengembangan program yang lebih efisien

BAB II

PEMBAHASAN

2.1 Pengertian Singel Linked List

Singel linked list Adalah struktur data yang terdiri dari Kumpulan node (simpul) yang saling terhubung secara berurutan. Setiap node memiliki dua bagian utama, yaitu:

- Data: menyimpan nilai atau informasi
- Next (pointer/reference): menunjuk ke node berikutnya

Pada single linked list, setiap node hanya memiliki satu arah hubungan, yaitu ke node berikutnya. Node terakhir akan menunjuk nilai null, yang menandakan akhir dari list.

2.2 Alat dan Bahan

1. Laptop
2. Aplikasi aclipse
3. Paket data yang stabil

2.3 Langkah-langkah

A. Class NodeSLL_2511533015

1. Kode program

```
package pekan5_2511533015;

public class NodeSLL_2511533015 {
    // node bagian data
    int data_3015;
    // pointer ke node berikutnya
    NodeSLL_2511533015 next_3015;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511533015 (int data)
    {
        this.data_3015= data;
        this.next_3015 = null;
    }
}
```

2. Penjelasan

- a. Deklarasi package
`package pekan5_2511533015;`
 - Baris ini menunjukkan bahwa kelas tersebut berada dalam package Bernama `pekan5_2511533015`
 - Package digunakan untuk mengelompokkan class agar lebih terorganisir
- b. Deklarasi class
`public class NodeSLL_2511533015 {`
 - Ini Adalah deklarasi class Bernama `NodeSLL_2511533015`
 - Class ini berfungsi sebagai representasi satu node (simpul) dalam struktur Single Linked List
- c. Variabel data (atribut node)
`int data_3015;`
 - Variabel ini digunakan untuk menyimpan nilai/data dalam node
 - Tipe data `int` berarti hanya menyimpan angka bilangan bulat
- d. Pointer ke node berikutnya
`NodeSLL_2511533015 next_3015;`
 - Variabel ini berfungsi sebagai pointer/referensi ke node berikutnya
 - Tipe datanya Adalah class itu sendiri (`NodeSLL_2511533015`)
 - Ini yang membuat node saling terhubung dalam linked list
- e. Konstruktor
`public NodeSLL_2511533015 (int data)`
`{`
`this.data_3015 = data;`
`this.next_3015 = null;`
`}`
 - Konstruktor Adalah method khusus yang otomatis dipanggil saat objek dibuat
 - Parameter `int data` digunakan untuk mengisi nilai node
detail baris:
 - `This.data_3015 = data;`
 - Mengisi atribut `data_3015` dengan nilai yang diberikan saat objek dibuat
 - `This.next_3015 = null;`
 - Mengatur pointer `next_3015` menjadi null
 - Artinya node tersebut belum terhubung ke node lain (biasanya node terakhir)
- f. Kesimpulan fungsi class
Class ini digunakan untuk:
 - Membuat node (simpul) dalam Single Linked List
 - Menyimpan:
 - Data (nilai)
 - Alamat node berikutnya

3. output

```
1
2 <terminated> HapusSLL_2511533015 [Java Application] C:\Users\use
3 list awal:
4 1-->2-->3-->4-->5-->6
5 List setelah head dihapus:
6 2-->3-->4-->5-->6
7 List setelah simpul terakhir di hapus:
8 2-->3-->4-->5
9 List setelah posisi 2 dihapus:
0 2-->4-->5
1
2
3
4
```

B. Class TambahSLL_2511533015

1. Kode program

```
package pekan5_2511533015;
public class TambahSLL_2511533015 {
    public static NodeSLL_2511533015 insertAtFront_3015 (NodeSLL_2511533015
head_3015,int value_3015) {
        NodeSLL_2511533015 new_node_3015 = new NodeSLL_2511533015
(value_3015);
        new_node_3015.next_3015 = head_3015;
        return new_node_3015;
    }
    //fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511533015 insertAtEnd_3015 (NodeSLL_2511533015
head_3015, int value_3015) {
        //buat sebuah node dengan sebuah nilai
        NodeSLL_2511533015 newNode_3015 = new NodeSLL_2511533015
(value_3015);
        //jika list kosong maka node jadi head
        if (head_3015 == null) {
            return newNode_3015;
        }
        //simpan head ke variabel sementara
        NodeSLL_2511533015 last_3015 = head_3015;
        //telusuri ke node akhir
        while (last_3015.next_3015 != null) {
            last_3015 = last_3015.next_3015;
        }
        //ubah pointer
        last_3015.next_3015 = newNode_3015;
        return head_3015;
    }
}
```



```

        static NodeSLL_2511533015 GetNode(int data_3015) {
            return new NodeSLL_2511533015 (data_3015);
        }
        static NodeSLL_2511533015 insertPos (NodeSLL_2511533015
headNode_3015, int position_3015, int value_3015) {
    NodeSLL_2511533015 head_3015 = headNode_3015;
    if (position_3015 < 1)
        System.out.print ("Invalid position");
    if (position_3015 == 1) {
        NodeSLL_2511533015 new_node_3015 = new
NodeSLL_2511533015 (value_3015);
        new_node_3015.next_3015 = head_3015;
        return new_node_3015;
    } else {
        while (position_3015-- != 0) {
            if (position_3015 == 1) {
                NodeSLL_2511533015 newNode_3015 =
GetNode (value_3015);
                newNode_3015.next_3015 =
headNode_3015.next_3015;
                headNode_3015.next_3015 = newNode_3015;
                break;
            }
            headNode_3015 = headNode_3015.next_3015;
        }
        if (position_3015 != 1)
            System.out.print("Posisi di luar jangkauan");
    }
    return head_3015;
}

public static void printList (NodeSLL_2511533015 head_3015)
{
    NodeSLL_2511533015 curr = head_3015;
    while (curr.next_3015 != null) {
        System.out.print (curr.data_3015+"-->");
        curr = curr.next_3015;
    }
    if (curr.next_3015==null) {
        System.out.print(curr.data_3015);
        System.out.println();
    }
    public static void main (String [] args) {
        //buat linked list 2->3->4->5->6
        NodeSLL_2511533015 head = new
NodeSLL_2511533015 (2);
        head.next_3015 = new NodeSLL_2511533015
(3);
        head.next_3015.next_3015 = new
NodeSLL_2511533015 (5);
        head.next_3015.next_3015.next_3015 = new
NodeSLL_2511533015 (6);
    }
}

```

```

//cetak list asli
System.out.print("Senarai berantai awal: ");
printList (head);
//tambahkan node baru didepan
System.out.print("tambah 1 simpul di depan: ");
int data_3015 = 1;
head= insertAtFront_3015 (head, data_3015);
//cetak update list
printList (head);
//tambahkan node baru di belakang
System.out.print ("tambah 1 simpul di belakang:
");

int data2 = 7;
head = insertAtEnd_3015 (head, data2);
//cetak update list
printList (head);
System.out.print("tambah 1 simpul ke data 4:");
int data3 = 4;
int pos=4;
head = insertPos (head,pos,data3);
//cetak update list
printList (head);
}
}

```

2. Penjelasan

a. Struktur umum program

```
package pekan5_2511533015;
```

```
public class TambahSLL_2511533015 {
```

- Program ini berada dalam package pekan5_2511533015
- Class ini berfungsi untuk menambahkan node pada Single Linked List

b. Fungsi tambah di depan

```
public static NodeSLL_2511533015 insertAtFront_3015 (NodeSLL_2511533015
head_3015,int value_3015)
```

cara kerja:

```
NodeSLL_2511533015 new_node_3015 = new NodeSLL_2511533015 (value_3015);
```

```
new_node_3015.next_3015 = head_3015;
```

```
return new_node_3015;
```

- Membuat node baru dengan nilai value_3015
- Node baru diarahkan ke head lama
- Node baru menjadi head yang baru

Contoh:

Awal: 2 → 3 → 5 → 6

Tambah depan 1

Hasil: 1 → 2 → 3 → 5 → 6

c. Fungsi tambah di akhir

```
public static NodeSLL_2511533015 insertAtEnd_3015 (NodeSLL_2511533015
head_3015, int value_3015)
```

- Cara kerja:
 - Buat node baru
 - Jika list kosong -> jadi head
 - Jika tidak:
 - Telusuri sampai node terakhir
 - Sambungkan node terakhir ke node baru

- Contoh:

1 → 2 → 3 → 5 → 6

Tambah belakang 7

Hasil: 1 → 2 → 3 → 5 → 6 → 7

d. Fungsi membuat node

```
static NodeSLL_2511533015 GetNode(int data_3015)
```

- Fungsi sederhana untuk membuat node baru
- Mengembalikan node dengan isi data tertentu

e. Fungsi tambah di posisi tertentu

```
static NodeSLL_2511533015 insertPos (...)
```

tujuan:

menambahkan node pada posisi tertentu

penjelasan alur:

- jika posisi = 1
 - new_node.next = head;
 - return new_node;
- # Sama seperti tambah di depan
- jika posisi selain 1
 - while (position_3015-- != 0)

loop berjalan sampai posisi yang diinginkan

Ketika posisi sudah tepat:

```
newNode.next = headNode.next;
```

```
headNode.next = newNode;
```

artinya: node baru disisipkan di Tengah

- jika posisi tidak valid
 - System.out.print("Posisi di luar jangkauan");
- Contoh:
 - 1 → 2 → 3 → 5 → 6 → 7
 - Tambah 4 di posisi ke-4
 - Hasil: 1 → 2 → 3 → 4 → 5 → 6 → 7

f. fungsi cetak linked list

```
public static void printList (NodeSLL_2511533015 head_3015)
```

cara kerja:

```
while (curr.next_3015 != null)
```

```
# menelusuri dari awal sampai akhir
```

```
# menampilkan data dengan format:
```

```
data --> data --> data
```

contoh output:

```
1-->2-->3-->4-->5-->6-->7
```

g. fungsi main (program utama)

- membuat linked list awal

```
2 → 3 → 5 → 6
```

- cetak awal

```
Senarai berantai awal: 2-->3-->5-->6
```

- tambah di depan

```
insertAtFront(1)
```

hasil:

```
1 → 2 → 3 → 5 → 6
```

- tambah di belakang

```
insertAtEnd(7)
```

hasil:

```
1 → 2 → 3 → 5 → 6 → 7
```

- tambah di posisi ke-4

```
insertPos(4, 4)
```

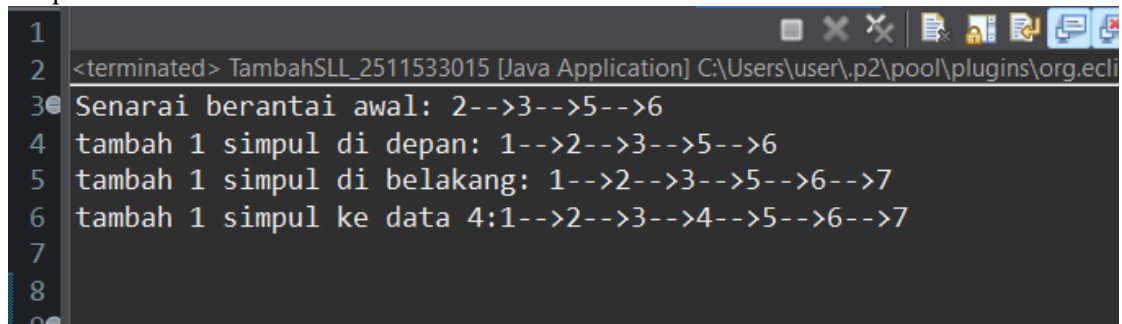
hasil:

```
1 → 2 → 3 → 4 → 5 → 6 → 7
```

h. Kesimpulan program

- Menambah node di depan
- Menambah node di belakang
- Menambah node di posisi tertentu
- Menampilkan isi linked list

3. Output



C. PencarianSLL_2511533015

1. Kode program

```
package pekan5_2511533015;  
  
public class PencarianSLL_2511533015 {
```

```

static boolean searchKey (NodeSLL_2511533015 head, int key) {
    NodeSLL_2511533015 curr = head;
    while (curr != null) {
        if (curr.data_3015 == key)
            return true;
        curr = curr.next_3015;
    }
    return false;
}

public static void traversal (NodeSLL_2511533015 head) {
    //mulai dari head
    NodeSLL_2511533015 curr = head;
    //telusuri sampai pointer null
    while (curr != null) {
        System.out.print (" " + curr.data_3015);
        curr = curr.next_3015;
    }
    System.out.println ();
}

public static void main (String[] args) {
    // TODO Auto-generated method stub
    NodeSLL_2511533015 head = new NodeSLL_2511533015 (14);
    head.next_3015 = new NodeSLL_2511533015 (21);
    head.next_3015.next_3015 = new NodeSLL_2511533015 (13);
    head.next_3015.next_3015.next_3015 = new NodeSLL_2511533015
(30);
    head.next_3015.next_3015.next_3015.next_3015 = new
NodeSLL_2511533015 (10);
    System.out.print ("Penelusuran SLL: ");
    traversal (head);
    // data yang akan dicari
    int key = 30;
    System.out.print ("cari data " + key + " - ");
    if (searchKey (head, key))
        System.out.println ("ketemu");
    else
        System.out.println ("tidak ada");
}
}

```

2. Penjelasan

- a. Deklarasi package dan class


```
package pekan5_2511533015;
```

```
public class PencarianSLL_2511533015 {
```

- Program berada dalam package pekan5_2511533015
- Class ini digunakan untuk:
 - Menelusuri (traversal) linked list
 - Mencari data (searching) dalam linked list

- b. Fungsi pencarian data

```
static boolean searchKey (NodeSLL_2511533015 head, int key)
```

tujuan:

- Mencari apakah suatu nilai (key) ada di dalam linked list

Cara kerja:

```
NodeSLL_2511533015 curr = head;
```

➤ Mulai dari node pertama (head)

```
while (curr != null ) {
```

➤ Loop berjalan selama node belum habis

```
if (curr.data_3015 == key)
```

```
return true;
```

➤ Jika data sama dengan yang dicari -> langsung berhenti dan return true

```
curr = curr.next_3015;
```

➤ Pindah ke node berikutnya

Jika tidak ditemukan:

```
return false;
```

Kesimpulan fungsi:

- Mengembalikan:
True=> jika data ditemukan
False=> jika data tidak ditemukan

c. Fungsi traversal (penelusuran)

```
public static void traversal (NodeSLL_2511533015 head)
```

tujuan:

- Menampilkan semua isi linked list

Cara kerja:

```
NodeSLL_2511533015 curr = head;
```

➤ Mulai dari head

```
while (curr != null) {
```

➤ Loop sampai akhir

```
System.out.print (" " + curr.data_3015);
```

➤ Cetak data

```
curr= curr.next_3015;
```

➤ Pindah ke node berikutnya

Output contoh:

14 21 13 30 10

d. Fungsi main (program utama)

- Membuat linked list
NodeSLL_2511533015 head = new NodeSLL_2511533015 (14);
head.next_3015 = new NodeSLL_2511533015 (21);
head.next_3015.next_3015 = new NodeSLL_2511533015 (13);
head.next_3015.next_3015.next_3015 = new NodeSLL_2511533015 (30);
head.next_3015.next_3015.next_3015.next_3015 = new NodeSLL_2511533015 (10);

struktur linked list:

14 → 21 → 13 → 30 → 10 → null

- Menampilkan data
System.out.print("Penelusuran SLL: ");
traversal (head);

output:

Penelusuran SLL: 14 21 13 30 10

- Proses pencarian
int key = 30;
➤ Data yang ingin dicari Adalah 30

if (searchKey (head, key))

⇒ Jika ditemukan:
cari data 30 – ketemu

⇒ Jika tidak :
cari data 30 - tidak ada

- e. Alur program secara sederhana

Buat Linked List

↓

Tampilkan isi list

↓

Cari data (key)

↓

Jika ada → "ketemu"

Jika tidak → "tidak ada"

- f. Kesimpulan program

Program ini berfungsi untuk:

- Menyimpan data dalam bentuk single Linked List
- Menampilkan seluruh isi list (traversal)
- Mencari data tertentu (searching)

3. Output

```
1
2 <terminated> PencarianSLL_2511533015 [Java Application]
3 Penelusuran SLL:  14 21 13 30 10
4 cari data 30 - ketemu
5
6
7
```

D. HapusSLL_2511533015

1. Kode program

```
package pekan5_2511533015;

public class HapusSLL_2511533015 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511533015 deleteHead (NodeSLL_2511533015
head_3015) {
        //jika SLL kosong
        if (head_3015 == null)
            return null;
        //pindahkan head ke node berikutnya
        head_3015 = head_3015.next_3015;
        //return head baru
        return head_3015;
    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511533015 removeLastNode (NodeSLL_2511533015
head_3015) {
        //jika list kosong, return null
        if (head_3015 == null) {
            return null;
        }
        //jika list satu node, hapus node dan return null
        if (head_3015.next_3015 == null) {
            return null;
        }
        //temukan node terakhir ke dua
        NodeSLL_2511533015 secondLast_3015 = head_3015;
        while (secondLast_3015.next_3015.next_3015 != null) {
            secondLast_3015 = secondLast_3015.next_3015;
        }
        //hapus node terakhir
        secondLast_3015.next_3015 = null;
        return head_3015;
    }
    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511533015 deleteNode (NodeSLL_2511533015
head_3015, int position_3015) {
```



```

NodeSLL_2511533015 temp = head_3015;
NodeSLL_2511533015 prev = null;
//jika linked list null
if (temp == null)
    return head_3015;
//kasus 1: head dihapus
if (position_3015 == 1) {
    head_3015 = temp.next_3015;
    return head_3015;}
//kasus 2: menghapus node di tengah
//telusuri ke node yang dihapus
for (int i = 1; temp != null && i < position_3015; i++) {
    prev = temp;
    temp = temp.next_3015;}
//jika ditemukan, hapus node
if (temp != null) {
    prev.next_3015 = temp.next_3015;
} else {
    System.out.println ("Data tidak ada");}
return head_3015;
}
//fungsi mencetak SLL
public static void printList (NodeSLL_2511533015 head_3015)
{
    NodeSLL_2511533015 curr = head_3015;
    while (curr.next_3015 != null) {
        System.out.print (curr.data_3015 + "-->");
        curr = curr.next_3015;}
    if (curr.next_3015==null) {
        System.out.print(curr.data_3015);
    }
    System.out.println ();}
//kelas main

public static void main(String[] args) {
    // TODO Auto-generated method stub
    //buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511533015 head = new NodeSLL_2511533015 (1);
    head.next_3015 = new NodeSLL_2511533015 (2);
    head.next_3015.next_3015 = new NodeSLL_2511533015 (3);
    head.next_3015.next_3015.next_3015 = new NodeSLL_2511533015 (4);
    head.next_3015.next_3015.next_3015.next_3015 = new
NodeSLL_2511533015 (5);
    head.next_3015.next_3015.next_3015.next_3015.next_3015 = new
NodeSLL_2511533015 (6);
    //cetak list awal
    System.out.println ("list awal: ");
    printList (head);
    //hapus head
    head = deleteHead (head);
}

```

```

        System.out.println ("List setelah head dihapus: ");
        printList (head);
        //hapus node terakhir
        head = removeLastNode (head);
        System.out.println ("List setelah simpul terakhir di hapus: ");
        printList (head);
        //deleting node at position 2
        int position = 2;
        head = deleteNode (head, position);
        //print list after deletion
        System.out.println ("List setelah posisi 2 dihapus: ");
        printList (head);
    }
}

```

2. Penjelasan

- a. Deklarasi package dan class
package pekan5_2511533015;

public class HapusSLL_2511533015 {
penjelasan:

- Program berada pada package pekan5_2511533015
- Class ini digunakan untuk:
 - Menghapus node depan
 - Menghapus node belakang
 - Menghapus node pada posisi tertentu
 - Menampilkan isi linked list

- b. Fungsi menghapus head

public static NodeSLL_2511533015 deleteHead (NodeSLL_2511533015 head_3015)

tujuan:

menghapus node pertama (head)

cara kerja:

if (head_3015 == null)

return null;

- Jika linked list kosong => return null

head_3015 = head_3015.next_3015;

- Head dipindahkan ke node berikutnya
- Node lama otomatis terhapus

return head_3015;

- Mengembalikan head baru

Contoh:

Awal:

1 → 2 → 3 → 4

Hapus head:

2 → 3 → 4

c. Fungsi menghapus node terakhir

```
public static NodeSLL_2511533015 removeLastNode (NodeSLL_2511533015  
head_3015)
```

tujuan:

menghapus node paling akhir

cara kerja:

- Jika list kosong
if (head_3015 == null)
 - Return null
- Jika hanya 1 node
if (head_3015.next_3015 == null)
 - Node dihapus dan return null
- Mencari node kedua terakhir
while (secondLast_3015.next_3015.next_3015 != null)
 - Loop berhenti di node sebelum terakhir
- Menghapus node terakhir
secondLast_3015.next_3015 = null;
contoh:
1 → 2 → 3 → 4 → 5
Menjadi:
1 → 2 → 3 → 4

d. Fungsi menghapus node di posisi tertentu

```
public static NodeSLL_2511533015 deleteNode (...)
```

tujuan:

menghapus node berdasarkan posisi

- Penjelasan variable
NodeSLL_2511533015 temp = head_3015;
NodeSLL_2511533015 prev = null;
- Temp => node yang sedang dicek
- Prev => node sebelumnya
 - Kasus 1: list kosong
if (temp == null)
return head_3015;
 - Kasus 2: menghapus head
if (position_3015 == 1)
cara kerja:
head_3015 = temp.next_3015;
⇒ Head dipindahkan
 - Kasus 3: menghapus node Tengah
for (int i = 1; temp != null && i < position_3015; i++)

- Menelusuri sampai posisi yang dicari

```
prev = temp;
temp = temp.next_3015;
```
 - Prev menyimpan node sebelumnya
 - Temp bergerak maju
 - Menghapus node

```
prev.next_3015 = temp.next_3015;
```

 artinya:
 - Node sebelumnya langsung menunjuk ke node setelah temp
 - Node temp terlepas dari linked list
 Contoh:
 $1 \Rightarrow 2 \Rightarrow 3 \Rightarrow 4 \Rightarrow 5$
 Hapus posisi 3
 Hasil:
 $1 \Rightarrow 2 \Rightarrow 4 \Rightarrow 5 \Rightarrow$
- e. Fungsi menampilkan linked list

```
Public static void printList (NodeSLL_2511533015 head_3015)
```

 Cara kerja:

```
While (curr.next_3015 != null)
```

 - Menelusuri node satu per Satu

```
System.out.print (curr.data_3015 + "→");
```

 Output:
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
- f. Fungsi main (program utama)
- Membuat linked list
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$
 - Menampilkan list awal

```
printList(head);
```

 output:
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$
 - Menghapus head

```
Head = deleteHead(head);
```

 Hasil:
 $2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$
 - Menghapus node terakhir

```
Head = removeLastNode (head);
```

 Hasil:
 $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
 - Menghapus posisi ke-2

```
Head = deleteNode (head, position);
```

 Hasil:
 $2 \rightarrow 4 \rightarrow 5$
- g. Alur program

Buat linked list => tampilkan data => hapus head => hapus node terakhir => hapus node posisi tertentu => tampilkan hasil

h. Kesimpulan program

Program ini digunakan untuk:

- Menghapus node pertama
- Menghapus node terakhir
- Menghapus node pada posisi tertentu
- Menampilkan isi linked list

i. Kesimpulan singkat

- deleteHead () = menghapus node depan
- removeLastNode () = menghapus node belakang
- deleteNode () = menghapus node berdasarkan posisi
- printList () = menampilkan isi linked list

3. Output

```
1
2 <terminated> HapusSLL_2511533015 [Java Application] C:\Users\user\.p2
3 list awal:
4 1-->2-->3-->4-->5-->6
5 List setelah head dihapus:
6 2-->3-->4-->5-->6
7 List setelah simpul terakhir di hapus:
8 2-->3-->4-->5
9 List setelah posisi 2 dihapus:
10 2-->4-->5
11
12
13
```

BAB III

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

- Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:
- Impelementasi single linked list berhasil dilakukan menggunakan Bahasa pemrograman
 - Operasi dasar seperti penambahan, penghapusan dan penampilan data dapat berjalan dengan baik.
 - Struktur ini terbukti lebih fleksibel dibandingkan array dalam hal penambahan dan penghapusan data
 - Namun, akses data pada posisi tertentu membutuhkan traversal dari awal, sehingga kurang efisien dibandingkan array untuk akses langsung

3.2 Saran Pengembangan

- Untuk pengembangan lebih lanjut, beberapa saran yang dapat dilakukan Adalah:
- Mengembangkan program menjadi double linked list untuk memudahkan traversal dua arah
 - Menambahkan fitur pencarian (searching) dan pengurutan (sorting) data
 - Mengimplementasikan single linked list dalam studi kasus nyata, seperti sistem antrian atau manajemen data
 - Mengoptimalkan kode agar lebih efisien dan mudah dipahami
 - Menggunakan GUI (Graphical User Interface) agar tampilan program lebih interaktif

DAFTAR PUSTAKA

1. [Oracle Java Documentation](#). *Java Platform Standard Edition 8 Documentation*. Diakses pada 7 Mei 2026.
2. [GeeksforGeeks - Linked List Data Structure](#). *Linked List Data Structure*. Diakses pada 7 Mei 2026.
3. [W3Schools Java Tutorial](#). *Java Tutorial*. Diakses pada 7 Mei 2026.
4. [Programiz Java Linked List](#). *Linked List Data Structure*. Diakses pada 7 Mei 2026.
5. Malik, D. S. *Data Structures Using Java*. Boston: Cengage Learning, 2011.
6. Lafore, Robert. *Data Structures and Algorithms in Java*. Indianapolis: Sams Publishing, 2003.
7. Weiss, Mark Allen. *Data Structures and Algorithm Analysis in Java*. Boston: Pearson Education, 2012.
8. Munir, Rinaldi. *Struktur Data*. Bandung: Informatika, 2018.
9. Sedgewick, Robert dan Kevin Wayne. *Algorithms*. Boston: Addison-Wesley, 2011

TUGAS

1. Soal

1. Deskripsi Tugas

Pada tugas ini, mahasiswa diminta untuk mengimplementasikan konsep Single Linked List yang bekerja secara dinamis menggunakan pointer (referensi antar node). Tema tugas kali ini adalah Simulasi Antrian Rumah Sakit. Mahasiswa diwajibkan membuat program yang mensimulasikan sistem pendaftaran antrian pasien, di mana setiap pasien direpresentasikan sebagai sebuah node yang memiliki pointer ke pasien berikutnya. Pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil (FIFO — First In, First Out).

2. Jawaban

➔ Class Pasien_2511533015

A. Kode program

```
package tugaspekan5_2511533015;

public class Pasien_2511533015 {
    //atribut pasien
    private String namaPasien_3015;
    private String keluhan_3015;
    private int nomorAntrian_3015;

    //pointer ke node berikutnya
    Pasien_2511533015 next_3015;

    //konstruktor
    public Pasien_2511533015 (String namaPasien_3015,String
keluhan_3015, int nomorAntrian_3015) {
        this.namaPasien_3015 = namaPasien_3015;
        this.keluhan_3015 = keluhan_3015;
        this.nomorAntrian_3015 = nomorAntrian_3015;
        this.next_3015 = null;
    }

    //getter
    public String getNamaPasien_3015() {
        return namaPasien_3015;
    }

    public String getKeluhan_3015() {
        return keluhan_3015;
    }
}
```



```

    }

    public int getNomorAntrian_3015() {
        return nomorAntrian_3015;
    }

    public Pasien_2511533015 getNext_3015() {
        return next_3015;
    }

    //setter
    public void setNamaPasien_3015 (String namaPasien_3015) {
        this.namaPasien_3015 = namaPasien_3015;
    }

    public void setKeluhan_3015 (String keluhan_3015) {
        this.keluhan_3015 = keluhan_3015;
    }

    public void setNomorAntrian_3015 (int nomorAntrian_3015) {
        this.nomorAntrian_3015 = nomorAntrian_3015;
    }

    public void setNext_3015 (Pasien_2511533015 next_3015) {
        this.next_3015 = next_3015;
    }
}

```

B. Penjelasan

1. Deklarasi package

Package tugaspekan5_2511533015;

Penjelasan:

- Kode ini berada di dalam package Bernama Tugaspekan5_2511533015
- Package digunakan untuk mengelompokkan class agar program lebih rapi dan terstruktur

2. Deklarasi class

Public class pasien_2511533015{

Penjelasan:

- Membuat class Bernama: Pasien_2511533015
 - Class ini berfungsi sebagai:
 - Representasi data pasien
 - Node pada single linked list
- Artinya:
Setiap objek pasien akan menjadi satu simpul/node dalam antrian rumah sakit.

3. Atribut pasien
private String namaPasien_3015;
private String keluhan_3015;
private int nomorAntrian_3015;
penjelasan:

- j. namaPasien_3015
private String namaPasien_3015;
- menyimpan nama pasien
 - bertipe string
- contoh:
“aras”
- k. keluhan_3015
private String keluhan_3015;
- menyimpan keluhan atau penyakit pasien
- contoh:
“Jatuh”
- l. nomorAntrian_3015
private int nomorAntrian_3015;
- menyimpan nomor antrian pasien
 - bertipe integer (int)
- contoh:
1, 2, 3, dst

4. pointer ke node berikutnya
Pasien_2511533015 next_3015;
Penjelasan:

- variable ini Adalah/referensi
 - digunakan untuk menunjuk pasien berikutnya dalam linked list
- ilustrasi:
Pasien1 → Pasien2 → Pasien3 → null
Setiap node menyimpan Alamat node berikutnya melalui:
next_3015

5. constructor
public Pasien_2511533015
(String namaPasien_3015,
String keluhan_3015,
int nomorAntrian_3015)

fungsi constructor:

- digunakan untuk menginisialisasi objek saat dibuat
- isi constructor
this.namaPasien_3015 = namaPasien_3015;
this.keluhan_3015 = keluhan_3015;
this.nomorAntrian_3015 = nomorAntrian_3015;
penjelasan:

#keyword:

this

➤ digunakan untuk membedakan:

- atribut milik class
- parameter constructor
 - #pointer awal
 - this.next_3015 = null;
- artinys:
 - saat node dibuat:
- belum terhubung ke node lain
- pointer masih kosong (null)

6. getter (selector)

getter digunakan untuk mengambil/membaca nilai atribut

a. getter nama pasien

```
public String getNamaPasien_3015() {  
    return namaPasien_3015;  
}
```

Fungsi:

- mengembalikan nama pasien

b. getter keluhan

```
public String getKeluhan_3015() {  
    return keluhan_3015;  
}
```

Fungsi:

- mengambil data keluhan pasien

c. getter nomor antrian

```
public int getNomorAntrian_3015() {  
    return nomorAntrian_3015;  
}
```

Fungsi:

- mengambil nomor antrian pasien

d. getter pointer next

```
public Pasien_2511533015 getNext_3015() {  
    return next_3015;  
}
```

Fungsi:

Mengambil node berikutnya

7. setter (mutator)

setter digunakan untuk mengubah isi atribut

a. setter nama pasien

```
public void setNamaPasien_3015 (String namaPasien_3015)  
fungsi:
```

- mengubah nama pasien

- b. setter keluhan
`public void setKeluhan_3015 (String keluhan_3015)`
 fungsi:
 - mengubah keluhan pasien
- c. setter nomor antrian
`public void setNomorAntrian_3015 (int nomorAntrian_3015)`
 fungsi:
 - mengubah nomor antrian pasien
- d. setter pointer next
`public void setNext_3015 (Pasien_2511533015 next_3015)`
 fungsi:
 - menghubungkan node ke node berikutnya

8. cara kerja class ini
 misalnya:
`Pasien_2511533015 p1 =`
`new Pasien_2511533015`
`("aras", "jatuh", 1);`
 Maka isi node:
 Nama Pasien : aras
 Keluhan : jatuh
 No Antrian : 1
 next : null

9. contoh menghubungkan node
`Pasien_2511533015 p2 =`
`new Pasien_2511533015`
`("si A", "sakit gigi", 2);`

`p1.next_3015 = p2;`
 struktur:
`[aras] → [siA] → null`

- ⇒ Kesimpulan program
 Class pasien_2511533015 digunakan untuk:
- Menyimpan data pasien
 - Menyimpan nomor antrian
 - Menyimpan keluhan pasien
 - Menghubungkan node satu dengan lainnya

Class ini menjadi dasar utama dalam pembuatan:
 Single Linked List Antrian Rumah Sakit

- ⇒ Inti penting
- Atribut = data pasien

- Next_3015 = penghubung antar node
- Constructor = membuat node baru
- Getter = mengambil data
- Setter = mengubah data

➔ Class RumahSakit_2511533015

A. Kode program

```
package tugaspekan5_2511533015;
import java.util.Scanner;
public class RumahSakit_2511533015 {

    static Pasien_2511533015 head_3015 = null;
    static int counter_3015 = 0;

    //insert at tail
    public static void daftarkanPasien_3015 (String nama_3015,
String keluhan_3015) {
        counter_3015++;

        Pasien_2511533015 pasienBaru_3015 = new
Pasien_2511533015 (nama_3015, keluhan_3015, counter_3015);

        //jika ;ist kosong
        if (head_3015 == null) {
            head_3015 = pasienBaru_3015;
        } else {

            //traversal ke node terakhir
            Pasien_2511533015 temp_3015 = head_3015;

            while (temp_3015.next_3015 != null) {
            }

            //sambungkan node terakhir ke node baru
            temp_3015.next_3015 = pasienBaru_3015;
        }

        System.out.println ("Pasien berhasil didaftarkan!");
        System.out.println ("Nomor Antrian: " +
pasienBaru_3015.getNomorAntrian_3015());
    }

    //delete head
    public static void panggilPasien_3015() {

        //cek list kosong
        if (head_3015 == null) {
            System.out.println ("Antrian kosong!");
        }
    }
}
```

```

        return;
    }

    System.out.println ("\nPasien Dipanggil:");
    System.out.println ("Nomor Antrian : " +
head_3015.getNomorAntrian_3015());
    System.out.println ("Nama Pasien : " +
head_3015.getNamaPasien_3015());
    System.out.println ("Keluhan : " +
head_3015.getKeluhan_3015());

    //geser head
    head_3015 = head_3015.next_3015;
}

//display
public static void tampilkanAntrian_3015() {
    if (head_3015 == null) {
        System.out.println ("Antrian kosonbg!");
        return;
    }

    Pasien_2511533015 temp_3015 = head_3015;

    System.out.println ("\n=== DAFTAR ANTRIAN PASIEN
===");

    while (temp_3015 != null) {

        System.out.println ("-----
---");

        System.out.println ("Nomor Antrian : " +
temp_3015.getNomorAntrian_3015());
        System.out.println ("Nama Pasien : " +
temp_3015.getNamaPasien_3015());
        System.out.println ("Keluhan : " +
temp_3015.getKeluhan_3015());

        temp_3015 =temp_3015.next_3015;
    }
}

//search
public static void cariPasien_3015 (String namaCari) {
    if (head_3015 == null) {
        System.out.println ("Antrian kosong!");
        return;
    }
}

```

```

        Pasien_2511533015 temp_3015 = head_3015;
        boolean ditemukan = false;

        while (temp_3015 != null) {

            //case-insensitive
            if
(temp_3015.getNamaPasien_3015().equalsIgnoreCase(namaCari)) {

                System.out.println ("\nPasien
Ditemukan!");

                System.out.println ("Nomor Antrian : " +
temp_3015.getNomorAntrian_3015());
                System.out.println ("Nama Pasien : " +
temp_3015.getNamaPasien_3015());
                System.out.println ("Keluhan : " +
temp_3015.getKeluhan_3015 ());

                ditemukan = true;
                break;
            }

            temp_3015 = temp_3015.next_3015;
        }

        if (!ditemukan) {
            System.out.println ("Pasien tidak ditemukan!");
        }
    }

    //cek status antrian
    public static void cekStatusAntrian_3015() {

        if (head_3015 == null) {
            System.out.println("Antrian kosong!");
            return;
        }

        int jumlah = 0;

        Pasien_2511533015 temp_3015 = head_3015;

        while (temp_3015 != null) {
            jumlah++;
            temp_3015 = temp_3015.next_3015;
        }

        System.out.println("\n=== STATUS ANTRIAN ===");
        System.out.println("Jumlah Pasien : " + jumlah);
    }

```

```

        System.out.println("\nPasien Terdepan:");
        System.out.println("Nomor Antrian : " +
head_3015.getNomorAntrian_3015());
        System.out.println("Nama Pasien : " +
head_3015.getNamaPasien_3015());
        System.out.println("Keluhan : " +
head_3015.getKeluhan_3015());
    }

    //main program
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        int pilihan;

        do {

            System.out.println("\n=== Antrian Rumah Sakit NIM:
2511533015 ===");
            System.out.println("1. Daftarkan Pasien (Insert)");
            System.out.println("2. Panggil Pasien (Delete
Head)");
            System.out.println("3. Tampilkan Antrian (Display)");
            System.out.println("4. Cari Pasien (Search)");
            System.out.println("5. Cek Status Antrian");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:

                    System.out.print("Masukkan Nama Pasien : ");
                    String nama_3015 = input.nextLine();

                    System.out.print("Masukkan Keluhan : ");
                    String keluhan_3015 = input.nextLine();

                    daftarkanPasien_3015(nama_3015,
keluhan_3015);

                    break;
                case 2:
                    panggilPasien_3015();

```



```

        break;

        case 3:
            tampilkanAntrian_3015();
            break;

        case 4:
            System.out.print("Masukkan Nama Pasien yang
dicari : ");
            String cari = input.nextLine();
            cariPasien_3015(cari);
            break;

        case 5:
            cekStatusAntrian_3015();
            break;

        case 6:
            System.out.println("Program selesai.");
            break;

        default:
            System.out.println("Pilihan tidak valid!");
    }

    } while (pilihan != 6);

    input.close();
}

```

B. Penjelasan

1. deklarasi package dan import

```

package tugaspekan5_2511533015;
import java.util.Scanner;

```

penjelasan:

- Program berada dalam package tugaspekan5_2511533015
- Scanner digunakan untuk menerima input dari keyboard

2. Deklarasi class

```

public class RumahSakit_2511533015 {
    fungsi class:

```

- Class ini berfungsi sebagai:
 - Driver program
 - Pengelola antrian pasien menggunakan single linked list

3. variable global


```
static Pasien_2511533015 head_3015 = null;
static int counter_3015 = 0;
```

 - a. head_3015


```
static Pasien_2511533015 head_3015 = null;
```

 fungsi:
 - menyimpan node pertama (head)
 - awalnya bernilai null
 - ⇒ artinya linked list masih kosong
 - b. counter_3025


```
static int counter_3015 = 0;
```

 fungsi:
 - menyimpan nomor antrian terakhir
 - akan bertambah otomatis saat pasien baru ditambahkan
4. method daftarkan pasien


```
public static void daftarkanPasien_3015
(String nama_3015, String keluhan_3015)
Fungsi:
```

 - menambahkan pasien baru di akhir linked list

Langkah-langkahnya

- a. counter bertambah


```
counter_3015++;
```

 fungsi:
 - nomor antrian otomatis bertambah
- b. membuat node pasien baru


```
Pasien_2511533015 pasienBaru_3015 =
new Pasien_2511533015
(nama_3015, keluhan_3015, counter_3015);
```

 Fungsi:
 - membuat node baru berisi:
 - nama pasien
 - keluhan
 - nomor antrian
- c. jika linked list kosong


```
if (head_3015 == null)
```

 fungsi:
 - jika belum ada pasien:
 - ⇒ node baru langsung menjadi head
- d. traversal ke node terakhir


```
Pasien_2511533015 temp_3015 = head_3015;
```

- digunakan untuk penelusuran
 - e. sambungkan node baru
 temp_3015.next_3015 = pasienBaru_3015;
 fungsi:
 - node terakhir menunjuk ke pasien baru
5. method panggil pasien
 public static void panggilPasien_3015()
 fungsi:
- menghapus pasien paling depan
 - a. cek list kosong
 if (head_3015 == null)
 jika kosong:
 Antrian kosong!
 - b. Menampilkan pasien yang dipanggil
 System.out.println ("Nomor Antrian : " + head_3015.getNomorAntrian_3015());
 Menampilkan:
 ➤ Nomor antrian
 ➤ Nama pasien
 ➤ Keluhan
 - c. Geser head
 head_3015 = head_3015.next_3015;
 fungsi:
 ➤ Node pertama dihapus
 ➤ Head berpindah ke node berikutnya
6. Method tampilkan antrian
 public static void tampilkanAntrian_3015()
 fungsi:
- Menampilkan seluruh isi linked list
 - a. Jika kosong
 if (head_3015 == null)
 - b. Traversal
 while (temp_3015 != null)
 fungsi:
 ○ Menelusuri seluruh node
 - c. Menampilkan data pasien
 System.out.println ("Nomor Antrian : " + temp_3015.getNomorAntrian_3015());
 - d. Pindah node
 temp_3015 = temp_3015.next_3015;

7. Method cari pasien
public static void cariPasien_3015 (String namaCari)
fungsi:

- Mencari pasien berdasarkan nama
 - a. Traversal
while (temp_3015 != null)
 - b. Pencarian case-insensitive
equalsIgnoreCase(namaCari)
artinya:
 - Tidak membedakan huruf besar dan kecil
Contoh:
“Aras” = “aras”
 - c. Jika ditemukan
ditemukan = true;
break;
 - d. Jika tidak ditemukan
System.out.println ("Pasien tidak ditemukan!");

8. Method cek status antrian
public static void cekStatusAntrian_3015()
fungsi:

- Menampilkan:
 - Jumlah pasien
 - Pasien terdepan
- a. Menghitung jumlah pasien
while (temp_3015 != null)
counter:
jumlah++;
- b. Menampilkan pasien terdepan
Menggunakan:
head_3015

9. Method main (program utama)
public static void main(String[] args)
fungsi:

- Menjalankan menu program

10. Menu program

1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar

11. Switch case

switch (pilihan)

- Digunakan untuk menjalankan menu sesuai pilihan user

⇒ Contoh alur program

➤ Insert:

Input:

Nama : aras

Keluhan : jatuh

➤ Hasil:

1 → aras

➤ Insert lagi

2 → siA

➤ Linked list:

aras → siA → null

➤ Delete head

Pasien depan dipanggil:

aras dihapus

hasil:

siA → null

⇒ Konsep single linked list pada program

Program ini menggunakan:

Insert at Tail

Delete Head

Traversal

Searching

⇒ Kesimpulan program

Program ini digunakan untuk:

- Menambahkan pasien ke antrian
- Memanggil pasien terdepan
- Menampilkan seluruh antrian
- Mencari pasien berdasarkan nama
- Mengecek status antrian

⇒ Inti penting:

- Head → node pertama
- Next → penghubung antar node
- Insert → tambah node
- Delete head → hapus node depan
- Traversal → telusuri linked list

- Search → mencari data

3. Output

```
RumahSakit_2511533015 [Java Application] C:\Users\user\AppData\Local\Programs\org.eclipse.
=== Antrian Rumah Sakit NIM: 2511533015 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : aras
Masukkan Keluhan : jatuh
Pasien berhasil didaftarkan!
Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533015 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

Pasien Dipanggil:
Nomor Antrian : 1
Nama Pasien : aras
Keluhan : jatuh

=== Antrian Rumah Sakit NIM: 2511533015 ===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Antrian kosong!
```

=== Antrian Rumah Sakit NIM: 2511533015 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 1

Masukkan Nama Pasien : aras

Masukkan Keluhan : jatuh

Pasien berhasil didaftarkan!

Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533015 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 2

Pasien Dipanggil:

Nomor Antrian : 1

Nama Pasien : aras

Keluhan : jatuh

=== Antrian Rumah Sakit NIM: 2511533015 ===

1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 3

Antrian kosong!